

Java GUI Lab: Campus Tech Day Registration

CSC 191 - Object-Oriented Programming II - NetBeans Edition

Suggested time	75-100 minutes
Points	100 points + 10 bonus
IDE	Apache NetBeans

Lab Goal

Build a small desktop application with Java Swing. You will begin with a NetBeans project that already compiles and runs, then complete a sequence of TODO tasks. Each task adds a visible GUI feature so you can run the program often and immediately see the effect of your code.

Learning Objectives

- Create and organize Swing components such as **JFrame**, **JPanel**, **JLabel**, **TextField**, **JComboBox**, **JRadioButton**, **JCheckBox**, and **TextArea**.
- Use **BorderLayout**, **GridLayout**, and **GridBagLayout** to organize a user interface.
- Use **ButtonGroup** for mutually exclusive radio buttons and compare radio-button behavior with check boxes.
- Connect GUI controls to **ActionListener** event handlers.
- Read and validate user input, update an object, and display feedback to the user.
- Use **JOptionPane** to report input errors.
- Use NetBeans to navigate source packages, edit Java files, build the project, and run a GUI application.

What You Are Building

Campus Tech Day Registration - target layout

Registration Form

Name:

Email:

Class standing:

Attendance: ☒ In person
☐ Online

Workshops: ☒ Java GUI Design
☐ AI for Developers
☒ Computer Networking

Registration Summary

Name: Jordan Lee
Email: jordan@eku.edu
Class standing: Junior
Attendance: In person
Workshops:
- Java GUI Design
- Computer Networking

Status: Registration saved successfully.

Getting Started in NetBeans - Step by Step

1. Download the ZIP package from Canvas and **unzip** it first. Do not open files directly inside the ZIP.
2. Open **Apache NetBeans**. Choose **File > Open Project**.
3. Select the unzipped folder **CSC191_GUI_Lab_NetBeans_v4** and click **Open Project**.
4. In the **Projects** pane, expand the project, then expand **Source Packages** and the package **csc191guilab**.
5. You should now see **GuiLabApp.java**, **EventRegistrationFrame.java**, and **RegistrationModel.java**.
6. Before changing any code, right-click **GuiLabApp.java** and choose **Run File** (or press **Shift+F6**).
7. The starter window should appear with a placeholder panel on the left and a summary panel on the right. If it does, your setup is correct.
8. After each task, save the file and run again to test your changes.

Where to find the files

```
CSC191_GUI_Lab_NetBeans_v4
|-- src
|   |-- csc191guilab
|       |-- GuiLabApp.java
|       |-- EventRegistrationFrame.java
|       |-- RegistrationModel.java
|-- nbproject
|   |-- project.properties
|   |-- project.xml
|-- build.xml
|-- README.txt
|-- GUI_Lab_Instructions_NetBeans_v4.pdf
```

How to use each Java file

File	How to use it
GuiLabApp.java	This is the starting point of the program. Run this file to launch the GUI. You usually do not need to change this file.
EventRegistrationFrame.java	This is the main lab file . Most of your work happens here. Complete the numbered TODO tasks in this file in order.
RegistrationModel.java	This class stores the registration data. In Task 8, copy the validated values from the GUI into this object.
README.txt	Quick setup and compile/run instructions.
nbproject/ and build.xml	NetBeans/Ant project configuration files. Leave them alone unless your instructor tells you otherwise.

Important NetBeans rule for this lab

Use **Source** view and write the Swing code yourself. **Do not create a JFrame Form** and do not use the NetBeans drag-and-drop GUI Builder. The goal is to practice how Swing components, layout managers, and event listeners work in code.

Step-by-Step Tutorial: First 10 Minutes

Use this mini tutorial if you are not sure how to begin.

Step 1: Open the main lab file

In the Projects pane, double-click **EventRegistrationFrame.java**. Scroll through the file and find the TODO comments labeled Task 1, Task 2, and so on.

Step 2: Understand the existing structure

Notice that the constructor builds the window, sets a layout, and calls helper methods such as **buildHeaderPanel()**, **buildMainPanel()**, **configureComponents()**, and **wireEvents()**.

Step 3: Run before editing

Run **GuiLabApp.java** once. The starter version shows a placeholder on the left. That is normal. Task 1 asks you to replace that placeholder with the real form panel.

Step 4: Start with Task 1

In **buildMainPanel()**, replace the placeholder with a call to **buildFormPanel()**. Then in **buildFormPanel()**, change the panel layout to **GridBagLayout** and add a titled border.

Step 5: Continue with Tasks 2-4

Still inside **buildFormPanel()**, add the text fields, combo box, radio buttons, check boxes, and the two buttons.

Step 6: Task 5

Move to **configureComponents()**. Add tooltips, make Register the default button, and add one more usability improvement.

Step 7: Task 6

Open **wireEvents()** and connect the buttons with **addActionListener**. Then place your logic inside **handleRegister(...)** and **handleClear(...)**.

Step 8: Tasks 7-8

In **handleRegister(...)**, read the current GUI values, validate them, show an error dialog if needed, and if valid, copy the values into **RegistrationModel** and display a summary.

Step 9: Test often

After each visible change, save and run the program again. If something does not look right, check the component positions, layout constraints, or event-handler connection.

Checkpoint

By this point you should know **where the files are**, **which file you run**, **which file you edit most**, and **where the TODO tasks live**.

Part A - Build the Interface

Task 1 - Display and organize the form panel (15 pts)

- In **buildMainPanel()**, replace the placeholder panel with a call to **buildFormPanel()**.
- In **buildFormPanel()**, change the panel to use **GridBagLayout**.
- Add a titled border named **Registration Form**.
- Use the provided **gbc(x, y)** helper, or your own **GridBagConstraints**, to keep labels and controls aligned.

Checkpoint: The left half of the window now shows your form panel instead of the placeholder message.

Concept check: Why is a layout manager better than assigning fixed x/y coordinates to every component?

Task 2 - Add text and selection controls (15 pts)

- Add a label and the provided **nameField**.
- Add a label and the provided **emailField**.
- Add a label and the provided **classStandingBox**.
- Keep the labels aligned and make the controls easy to read.

Checkpoint: You can type a name and email and choose a class standing.

Task 3 - Add radio buttons for one-choice input (15 pts)

- Add **inPersonButton** and **onlineButton** to **attendanceGroup**.
- Place both radio buttons inside a small **JPanel**.
- Add that panel with the label **Attendance**.
- Run the application and verify that selecting one attendance mode automatically deselects the other.

Checkpoint: Only one attendance option can be active at a time.

Concept check: What does **ButtonGroup** change about radio button behavior?

Task 4 - Add check boxes and action buttons (15 pts)

- Place **javaWorkshopBox**, **aiWorkshopBox**, and **networkingWorkshopBox** in a vertical panel.
- Add that panel with the label **Workshops**.
- Add **registerButton** and **clearButton** in a small button panel near the bottom of the form.
- Make sure the interface is readable and no components overlap.

Checkpoint: The complete form is visible. You can select multiple workshops, but Register and Clear do not work yet.

Part B - Make the GUI Interactive

Task 5 - Improve GUI usability (10 pts)

- Complete **configureComponents()**.
- Add at least two useful tooltips with **setToolTipText(...)**.
- Make Register the default button with **getRootPane().setDefaultButton(registerButton)**.
- Make one additional small usability improvement of your choice.

Checkpoint: Pressing Enter activates Register when appropriate, and your interface includes at least two helpful tooltips.

Task 6 - Connect button events (15 pts)

- Complete **wireEvents()**.
- Connect Register to **handleRegister**.
- Connect Clear to **handleClear**.
- Use method references or lambdas. Keep the main logic inside the handler methods instead of writing all logic directly inside **wireEvents()**.

Checkpoint: Clicking each button reaches the correct event-handler method.

NetBeans tip: add a temporary **System.out.println("Register clicked")** line if you want to confirm the handler is firing.

Task 7 - Validate user input (10 pts)

- Inside **handleRegister(...)**, read the current values from the GUI components.
- Reject a blank name.
- Reject an email that does not contain both @ and a ..
- Reject the default **Select...** class standing.
- Reject the form if neither attendance radio button is selected.
- For invalid input, use **JOptionPane.showMessageDialog(...)** to show an error and stop the registration.

Checkpoint: Every invalid input is handled separately. A bad input should show an error instead of producing a registration summary.

Task 8 - Save, summarize, and clear (5 pts)

- For valid input, copy the values into the provided **RegistrationModel**.
- Build a readable multi-line summary and display it in **summaryArea**.
- Update **statusLabel** with a success message.
- Complete **handleClear(...)** so every input component, the model, the summary, and the status are reset.

Checkpoint: Valid data produces a complete summary; invalid data shows an error; Clear restores the initial state.

Bonus Challenge - Live Workshop Counter (+10 pts)

Add a label that shows how many workshops are currently selected (0-3). Update the number immediately whenever any workshop check box changes. Use an **ItemListener** or **ActionListener**; do not update the count only when Register is clicked.

Testing Checklist

- The project compiles and runs in NetBeans.
- `GuiLabApp.java` opens the application window.
- The registration form and summary panel are fully visible.
- Name and email accept text input.
- The class-standing combo box works.
- Only one attendance radio button can be selected at a time.
- Multiple workshop check boxes can be selected.
- Register calls the registration handler.
- Clear calls the clear handler.
- Blank name is rejected.
- Invalid email is rejected.
- Default class standing is rejected.
- Missing attendance choice is rejected.
- Valid input updates `RegistrationModel` and displays a readable summary.
- Clear resets all components and the model.
- Bonus: if attempted, the workshop count changes immediately as boxes are selected or cleared.

Debugging in NetBeans

- Red underline: hover over the code or look for the error icon in the editor margin.
- Build error: read the first meaningful error in the Output window; later errors may be caused by the first one.
- Program runs but the form is missing: confirm that **`buildMainPanel()`** calls **`buildFormPanel()`** after Task 1.
- Buttons do nothing: confirm **`wireEvents()`** is called by the constructor and that listeners were added.
- Radio buttons stay selected together: confirm both buttons were added to the same **`ButtonGroup`**.
- Layout looks cramped: inspect your **`GridBagConstraints`** values such as insets, fill, anchor, and grid positions.

Grading Rubric

Task	Evidence	Points
1	Form panel displayed; GridBagLayout and border used correctly	15
2	Text fields and combo box added and aligned	15
3	Radio buttons grouped and displayed correctly	15
4	Workshop check boxes and action buttons added	15
5	Tooltips, default button, and an additional usability improvement	10
6	Register/Clear listeners correctly connected to handler methods	15
7	Required validation and JOptionPane error feedback	10
8	Model updated, formatted summary, and complete reset behavior	5
Bonus	Live workshop counter updates on selection changes	+10

Before you submit

- Run the project one final time in NetBeans.
- Test both valid and invalid registrations.
- Test Clear after entering values in every control.
- Make sure your name is included in a comment near the top of **EventRegistrationFrame.java** if your instructor requests it.
- Close the running GUI before creating the submission ZIP.